

PROGETTAZIONE E IMPLEMENTAZIONE DI UN TOOL API-AGNOSTICO PER LA SECURITY ASSURANCE DI API REST

Enea Manzi – 65935A

Relatore

Prof. Marco Anisetti

Correlatore

Prof. Claudio Agostino Ardagna

Contesto e Gap. L'adozione delle architetture a microservizi cloud-native ha ridistribuito la complessità applicativa su un numero crescente di servizi autonomi comunicanti tramite API REST (Application Programming Interface Representational State Transfer), moltiplicando la superficie di attacco esposta in assenza di una governance sistematica del ciclo di vita degli endpoint. Il gateway API è emerso come punto di enforcement centralizzato: concentra politiche di autenticazione, autorizzazione e rate limiting in un piano di configurazione ispezionabile e costituisce il confine operativo tra client e servizi interni. Le vulnerabilità più rilevanti in questo contesto non si manifestano però come anomalie sintattiche: si presentano come richieste formalmente corrette che violano le regole di accesso del sistema, e richiedono la conoscenza del contratto dell'API per essere rilevate. Quattro delle prime cinque categorie di rischio nell'OWASP API Security Top 10 2023 [1] appartengono a questa classe semantica, che include violazioni di autorizzazione a livello di oggetto, difetti di autenticazione ed esposizione non intenzionale di dati.

Il panorama degli strumenti disponibili presenta quattro gap strutturali. Il primo (G1) è la cecità semantica degli scanner DAST (Dynamic Application Security Testing) e dei fuzzer generici: operano sulla sintassi delle richieste HTTP e non raggiungono le vulnerabilità semantiche, un'area affrontata da meno del 10% dei contributi nella letteratura di settore [2]. Il quarto (G4) è il problema dell'oracolo [3]: stabilire automaticamente se il comportamento osservato costituisce una violazione dipende dalla conoscenza del dominio del controllo specifico, e la costruzione sistematica di tali criteri è la limitazione strutturalmente meno affrontata nel testing automatico delle API REST. A questi si affiancano un compromesso ricorrente tra portabilità e profondità semantica negli strumenti specializzati (G2) e l'assenza di riproducibilità deterministica tra esecuzioni successive (G3), che preclude l'integrazione della verifica nei cicli di sviluppo continuativi.

Obiettivi. Ai quattro gap corrispondono quattro obiettivi di ricerca. O1 è progettare un tool contract-driven che adotti la specifica OpenAPI (OpenAPI Specification, OAS) come fondamento contrattuale per costruire probe semanticamente significative, organizzando le verifiche in una tassonomia strutturata di domini di sicurezza con riferimenti normativi tracciabili verso categorie OWASP e codici CWE (Common Weakness Enumeration). O2, conseguenza diretta di O1, è superare il compromesso portabilità-profondità: un sistema che deriva tutta la conoscenza del target da specifica e configurazione è applicabile a qualsiasi API REST documentata senza modifiche al codice. O3 è garantire la riproducibilità deterministica dell'output e verificarla empiricamente, condizione necessaria per l'integrazione nei cicli di sviluppo continuativi tramite exit code semantici. O4 è dimostrare che per ogni verifica di sicurezza implementata il criterio di valutazione può essere ricavato dalla conoscenza del dominio del controllo specifico e codificato a priori nella logica del test, applicandosi deterministicamente senza interpretazione manuale.

Metodologia. L'approccio metodologico poggia su quattro fondamentali. L'agnosticismo applicativo: specifica OAS e file di configurazione sono le uniche sorgenti di conoscenza del target a runtime, senza riferimenti hardcoded a nessun sistema specifico. Il paradigma contract-driven: la specifica guida la costruzione delle probe e fornisce il criterio di valutazione primario per le risposte strutturali. La gestione del problema dell'oracolo: per le verifiche che richiedono conoscenza del dominio, il criterio è ricavato dallo standard tecnico applicabile, dal confronto comportamentale tra identità di ruolo distinto o dall'ispezione della configurazione del gateway, e codificato a priori nella logica del test corrispondente secondo il modello degli *specified oracles*. La riproducibilità deterministica: oracoli fissi e parametrizzazione completa su specifica e configurazione rendono l'output indipendente dall'operatore e confrontabile tra esecuzioni successive.

Le verifiche sono organizzate in una tassonomia di otto domini di sicurezza (D0-D7) che coprono rispettivamente la discovery degli endpoint non documentati, l'autenticazione, l'autorizzazione, la validazione degli input, la configurazione del gateway, l'observability, l'hardening e i flussi di business logic. Il box gradient mappa le precondizioni di privilegio alla profondità dell'assessment: i test **BLACK_BOX** non richiedono credenziali, i test **GREY_BOX** richiedono credenziali applicative per ruoli distinti, i test **WHITE_BOX** richiedono accesso al piano di controllo del gateway; i tre livelli si adattano automaticamente alle precondizioni disponibili nel deployment. Uno scheduler basato su DAG (grafo aciclico diretto) risolve topologicamente le dipendenze tra test, garantendo la correttezza semantica dell'ordinamento di esecuzione e il determinismo della sequenza tramite tie-breaking lessicografico all'interno di ogni batch.

Implementazione. La realizzazione si articola in una pipeline a sette fasi con semantica di errore differenziata: le fasi di caricamento della configurazione, discovery della specifica OAS, costruzione dei contesti e costruzione del DAG sono bloccanti; le fasi di esecuzione dei test, teardown e generazione del report continuano in presenza di errori non fatali, producendo comunque un assessment parziale invece di interrompere silen-

ziosamente. Il meccanismo di discovery dinamico tramite ispezione dei package consente di aggiungere nuovi test creando un singolo file Python, senza registrazioni manuali nel core.

Il contratto tra il sistema e ogni verifica è definito da due classi base: **BaseTest** per i test nativi e **ExternalToolTest** per quelli che delegano l'analisi a strumenti esterni; ciascuna dichiara come metadati fissi il codice CWE, la categoria OWASP, la strategia di esecuzione e le dipendenze verso altri test, che costituiscono la sorgente da cui lo scheduler DAG costruisce il grafo. I tool esterni (Nuclei per la scansione di Shadow API, sslyze e testssl.sh per la configurazione TLS) sono integrati tramite una gerarchia di connector che isola il core dalle dipendenze esterne; la valutazione dei loro output è responsabilità esclusiva del test, non del connector. La type safety è garantita a doppio livello: mypy in modalità strict per l'analisi statica inter-modulo e Pydantic v2 per la validazione a runtime dei dati al confine con l'esterno. L'evidence store accumula i risultati in streaming su disco con sanitizzazione centralizzata delle credenziali; a completamento vengono prodotti tre output: archivio forense JSON, report HTML operativo e serializzazione strutturata per pipeline CI/CD (Continuous Integration/Continuous Deployment) con exit code semantici.

Validazione. La validazione è stata condotta su Forgejo 14.0.3, piattaforma di hosting Git con specifica OpenAPI generata nativamente, esposta tramite Kong 3.9 in modalità DB-less su un testbed locale basato su Docker. La scelta del target è motivata dalle sue proprietà strutturali: autenticazione effettiva con tre livelli di privilegio configurati, endpoint con diversi gradi di protezione e superfici di attacco concrete per tutti i domini attivi; il tool non contiene nessun riferimento specifico a Forgejo, confermando in modo concreto l'agnosticismo applicativo. Il testbed è stato progettato per produrre sia esiti positivi che negativi attesi: alcune configurazioni sono state deliberatamente rese non conformi per verificare la capacità del sistema di distinguere conformità e violazione.

La validazione si è articolata su quattro livelli: composizione del testbed, qualità ingegneristica della codebase, copertura funzionale e idempotenza, profilo computazionale. I 18 test attivi su 7 domini (D0-D4, D6-D7) hanno prodotto 9 PASS, 7 FAIL, 2 SKIP e 98 finding. Il sistema ha correttamente distinto conformità e violazione in tutti i casi attesi, rilevando anche una discrepanza reale tra le dichiarazioni della specifica e il comportamento effettivo del target: Forgejo dichiara globalmente il requisito di autenticazione su tutti gli endpoint, ma alcuni endpoint genuinamente pubblici rispondono con codice 2xx in assenza di credenziali. Due run indipendenti hanno restituito risultati byte-identici su tutti i contatori aggregati, con un delta sul wall-clock dello 0,23%, validando empiricamente il determinismo dichiarato. Il profilo computazionale ha mostrato un wall-clock di circa 290 secondi con un picco di memoria di 287-295 MB, compatibile con i vincoli tipici delle pipeline CI/CD.

Contributi, Limiti e Sviluppi Futuri. I contributi si articolano su due piani. Sul piano metodologico: la tassonomia a otto domini di security assurance, trasferibile indipendentemente dall'implementazione specifica; la formalizzazione del problema dell'ora-

colo nel security testing delle API REST con *specified oracles* derivati dalla conoscenza di dominio, distinguendo le sorgenti del criterio di valutazione in funzione del controllo da eseguire; il box gradient come mapping formale tra precondizioni di privilegio e profondità di copertura. Sul piano ingegneristico: un assessment engine deterministico basato su DAG con risoluzione topologica; una pipeline a sette fasi con semantica di errore differenziata; una gerarchia di connector per l'integrazione di tool esterni senza modifiche al core; exit code semantici per l'integrazione automatica in cicli di sviluppo continuativi.

I limiti strutturali del paradigma sono tre. Il principale è la dipendenza dalla qualità della specifica OAS: un contratto incompleto o disallineato rispetto all'implementazione reale produce un assessment parziale senza che il sistema possa rilevarlo. Nei deployment cloud managed l'assenza di adapter per i provider principali limita la copertura alle verifiche BLACK_BOX e GREY_BOX. Una terza tensione riguarda l'interfaccia del gateway adapter, che espone solo le funzionalità generalizzabili a tutti i gateway, lasciando le verifiche vendor-specific fuori dal perimetro corrente.

Le estensioni operative immediate riguardano il completamento della copertura del Dominio 5 (Observability), lo sviluppo dei connector per OFFAT e Schemathesis per la copertura completa del Dominio 2 (Authorization), l'implementazione degli adapter per i gateway cloud managed e la granularità degli exit code per livello di priorità. Le traiettorie di ricerca aperta comprendono la trasformazione del tool da monoprodotto a piattaforma di assurance modulare per protocolli eterogenei (GraphQL, gRPC, AsyncAPI), la parallelizzazione intra-batch con analisi formale della thread-safety sulle strutture condivise, e la coverage augmentation progressiva, che userebbe le osservazioni raccolte durante l'esecuzione per estendere dinamicamente la superficie di attacco nel corso della stessa run. Il lavoro dimostra che un assessment automatizzato, deterministico e integrabile nel ciclo di sviluppo è praticabile su target reali: la sicurezza cessa di essere un'osservazione puntuale e diventa una proprietà del processo, verificabile a ogni rilascio.

Riferimenti bibliografici

- [1] OWASP Foundation, "OWASP Top 10 API Security Risks – 2023," 2023. [Online]. Available: <https://owasp.org/API-Security/editions/2023/en/0x11-t10/>
- [2] A. Golmohammadi, M. Zhang, and A. Arcuri, "Testing RESTful APIs: A Survey," *ACM Transactions on Software Engineering and Methodology*, vol. 33, no. 1, pp. 27:1–27:41, Nov. 2023. [Online]. Available: <https://dl.acm.org/doi/10.1145/3617175>
- [3] E. T. Barr, M. Harman, P. McMinn, M. Shahbaz, and S. Yoo, "The Oracle Problem in Software Testing: A Survey," *IEEE Transactions on Software Engineering*, vol. 41, no. 5, pp. 507–525, May 2015. [Online]. Available: <https://ieeexplore.ieee.org/document/6963470>